Introdução à Aprendizagem Automática 2026/2027

Aprendizagem Não Supervisionada - Mini Projeto de Laboratório I

6 de outubro de 2026

1 Introdução

Este mini-projeto tem como objetivo a familiarização prática com os algoritmos de aprendizagem não supervisionada lecionados nas aulas teóricas. Em particular, é esperado que os alunos apliquem os conhecimentos adquiridos e os expandam seguindo um conceito de aprendizagem ativa, tendo por base um conjunto de dados reais. Este mini-projeto terá componentes de programação (implementação de algoritmos), aplicação dos algoritmos para visualização e interpretação/análise dos resultados. As aulas de laboratório servirão para acompanhar e avaliar de forma contínua a progressão dos projetos, mas é esperado que os alunos dediquem tempo ao trabalho fora das aulas de laboratório.

O trabalho deverá ser realizado em **grupos de 2 alunos**, sendo que cada grupo deve trabalhar sozinho. A linguagem de programação a usar será o **Python**. Recomenda-se a instalação de, pelo menos, as seguintes bibliotecas: *numpy, scipy, matplotlib, pandas, scikit-learn*. Alternativamente, os alunos poderão usar o **Google Colab**[*]como plataforma para implementarem todo o código do projeto. O Colab tem o benefício de que pode ser associado a uma conta de endereço Gmail, interliga com o Google Drive e não necessita de instalação das bibliotecas mais populares.

2 Submissão e Regras Sobre o Uso de IA

Cada grupo deve submeter um **ficheiro zip** no Fénix até dia **17 de Outubro de 2026**, com os seguintes ficheiros:

- Um **ficheiro Python**, idealmente um *Jupyter Notebook*, contendo todo o código desenvolvido para o projeto. Código que faça parte da avaliação, nomeadamente que corresponda a perguntas para escrita de funções, deve estar claramente indicado com comentários.
- O **PDF do enunciado preenchido** com as respostas às questões nos campos indicados - isto pode ser feito preenchendo diretamente o PDF num browser ou num leitor de PDF. Posteriormente, o PDF deve ser *flattened*/trancado, o que pode ser feito escolhendo a opção de imprimir o PDF e posteriormente guardar novamente apenas as páginas pretendidas.
- Todos os **gráficos/figuras pedidas**, identificadas com o nome da questão (*e.g.*, "PartI2a.jpg"). Os eixos e pontos devem ser sempre corretamente identificados.
- Os documentos do **compromisso de honra** e da declaração do uso de IA preenchidos e assinados. Ver templates dos mesmos nos anexos A e B.

Uso de IA: Os laboratórios de Introdução à Aprendizagem Automática têm uma política de "semáforo amarelo"quanto ao uso de IA. Permite-se o uso da mesma para auxílio na geração de código e na compreensão de bibliotecas e funções das mesmas. Adicionalmente, é permitido o uso de IA como suporte à aprendizagem ativa. **Todavia, os alunos deverão ser capazes de responder a questões feitas sobre o código durante as aulas, sendo isto contabilizado na componente da avaliação contínua. Não é permitido o uso de IA para gerar respostas.**

[*]https://colab.research.google.com/

3 Avaliação

A nota do mini-projeto I terá em conta o **código entregue**, as **respostas às questões** e a **avaliação contínua** a decorrer durante as duas sessões de laboratório dedicadas a este projeto. A avaliação contínua consistirá na interação com o docente de laboratório, sendo que a mesma poderá ter diferentes formatos. Este projeto terá um peso de 35% na nota final do **laboratório** da cadeira.

4 Dados

4.1 Descrição dos dados

O projeto tem como objetivo principal a análise não supervisionada de vídeos de fisioterapia de vários pacientes que sofreram acidentes vasculares cerebrais (AVC). Os pacientes foram gravados a fazer várias repetições de diferentes atividades diárias, cujo objetivo é promover a reabilitação motora. Os vídeos foram adquiridos usando uma câmera ZED da StereoLabs. Posteriormente recorreu-se ao MediaPipe[†], um modelo baseado em redes neuronais, para a deteção e extração de um conjunto de marcadores a cada 2 segundos de vídeo. Estes marcadores, como exemplificados na Figura 1, correspondem a posições 2D $(u_{j,t}^{(i)}, v_{j,t}^{(i)})$ de várias partes do corpo humano j e podem ser ligados por arestas, permitindo a formação de um esqueleto representativo da pose do paciente no dado instante t.

Formalmente, o conjunto de dados a ser usado consiste em sequências de poses de um número desconhecido de pacientes. Também não são conhecidos os tipos de exercícios que foram executados. Cada sequência $i \in \{0, .., 70\}$ contém um número variável de poses. Cada pose adquirida no instante t é representada por um conjunto de $j \in \{0, ..., 32\}$ marcadores, cada um deles definido por uma posição $(u_{j,t}^{(i)}, v_{j,t}^{(i)})$. Ou seja, dada uma sequência i, cada pose num instante t corresponde ao seguinte vetor: $[u_{0,t}^{(i)}, v_{0,t}^{(i)}, u_{1,t}^{(i)}, v_{1,t}^{(i)}, ..., u_{32,t}^{(i)}, v_{32,t}^{(i)}]$

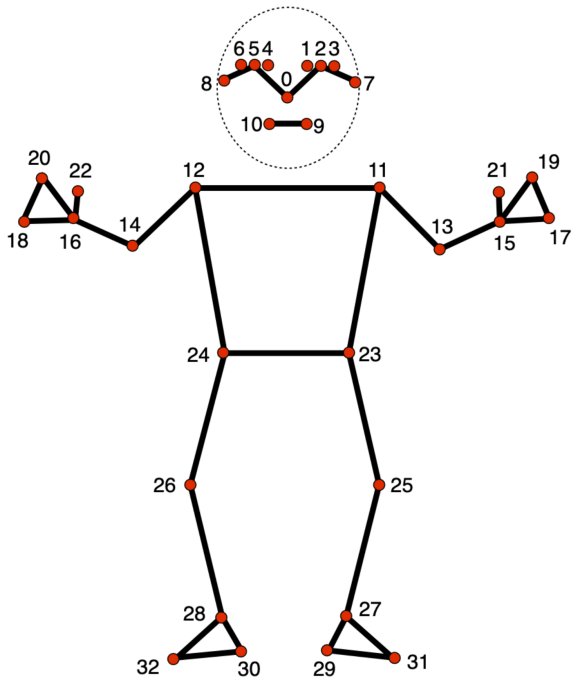


Figura 1: Esqueleto obtido com o MediaPipe Skeleton Keypoints. Cada número corresponde a um marcador e estes seguem a mesma ordem que no projeto.

[†]https://github.com/google-ai-edge/mediapipe

4.2 Upload e navegação preliminar dos dados

Os dados encontram-se guardados no formato pickle (.pkl), podendo ser integrados no código através da biblioteca *pandas*, que criará uma matriz estruturada de dados. O código abaixo providencia alguns exemplos sobre como navegar um:

Listing 1: Código de apoio à leitura e navegação de *dataframes*.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

#Importar os dados e verificar dimensões
df_train = pd.read_pickle('Xtrain.pkl')
print(df_train.shape)

#Concatenar todas as poses e sequências numa matriz de dados.
#Esta matriz vai ser usada na maior parte do projeto
X = np.concatenate(df_train['Skeleton_Sequence'].to_numpy())
print(X.shape) # N X 66

#Normalização dos dados - centrar + escalamento (recomendando e crítico para o PCA)
scaler = StandardScaler()
X_norm = scaler.fit_transform(X)

#Aceder a uma sequência específica do dataframe, por exemplo a primeira
seq = df_train.iloc[0]['Skeleton_Sequence']
print(seq.shape) # T X 66
```

Enunciado

Grupo: 109 **Turno:** IAAutomL06

Nome: David Alves **IST ID:** 107603

Nome: Luana Oliveira **IST ID:** 109251

Parte I - *k-means*

Esta secção tem como objetivo explorar o algoritmo *k-means*, a sua inicialização e optimização, assim como a aplicação do mesmo aos dados de fisioterapia.

1. **Implementação:** Modifique o código abaixo de forma a implementar o algoritmo *k-means*. O código deve permitir: i) a estimação da posição de K centróides definidos pelo utilizador; ii) a inicialização da posição dos mesmos usando o método aleatório ou o *k-means++*; iii) o cálculo da função de custo; e iv) a associação de novos pontos aos centróides estimados. As funções implementadas devem estar claramente identificadas no código entregue.

Listing 2: Código a modificar para a implementação do *k-means*.

```
import numpy as np

# NÃO MODIFICAR A SEED
np.random.seed(555)

def kminit(X, K, flag):
    #Inicialização - pode ser aleatória ou recorrendo ao k-means++

    '''
    Input:
    X: um numpy ndarray com dimensões (N,d), em que cada linha corresponde
    a uma amostra
    K: um inteiro que corresponde ao número de clusters a estimar
    flag: indicador sobre o tipo de inicialização. Sugestão: "random" e "km++"
    para cada versão

    Output:
    C: um numpy ndarray com dimensões (K,d), onde cada linha é a inicialização
    de um dos centróides
    '''

    # A inicialização do primeiro centróide é fixa
    N = X.shape[0]

    C = []
    C.append(X[np.random.randint(N)])

    # TODO: complete o código para inicializar os K-1 centróides de acordo com a flag

    return C


def kmeans_custo(X, C, s):
    #Calcula a função de custo do k-means

    '''
    Input:
    X: um numpy ndarray com dimensões (N,d), em que cada linha corresponde
    a uma amostra
```

```
    C: um numpy ndarray com dimensões  (K,d), onde cada linha é a incialização
    de um dos centróides
    s: um numpy ndarray de dimensão (N,) onde cada entrada é um inteiro que representa o
    indíce do cluster para o ponto x^i

    Returns squared Euclidean distances from each point to the center of its assigned cluster
    '''

    # TODO: calcule a função de custo

    return

def kmeans(X, K, flag):
    #Aplica o algoritm k-means à matriz de dados X.

    '''
    X: um numpy ndarray com dimensões (N,d), em que cada linha corresponde
    a uma amostra

    K: um inteiro que corresponde ao número de clusters a estimar

    flag: indicador sobre o tipo de inicialização. Sugestão: "random" e "km++"
    para cada versão

    Devolve um tuplo contendo (C, z):
        C: um numpy ndarray com dimensões  (K,d), onde cada linha é a incialização
        de um dos centróides
        z: um numpy ndarray de dimensão (N,) onde cada entrada é um inteiro
        que representa o indíce do cluster para
        o ponto x^(i)
    '''

    #TODO: Implemente o algoritmo do k-means, com um critério de paragem apropriado.
    #Use as funções definidas acima

    return
```

2. **Selecionar o número de centróides e inicialização**

   R2.a) Corra a implementação para os dados de fisioterapia, usando a inicialização aleatória e com um número variável de centróides K. Note que deverá usar uma matriz X que contenha todas as poses, independentemente da sequência. Crie um gráfico do valor função de custo em função do número de centróides, escolha o número óptimo e justifique a escolha.

   R2.b) Repita a questão anterior, mas agora utilizando a inicialização do *k-means++*.



R2.c) Corra 20 vezes cada uma das alíneas anteriores e represente num mesmo gráfico o valor médio da função de custo e o desvio padrão para cada número de centróides K e tipo de inicialização. Qual das inicializações aparenta ser melhor? Porquê?



3. **Visualização:** Escreva código que permita visualizar a pose associada a cada centróide. Use como referência a Figura 1. Não se esqueça de inverter a normalização para recuperar os dados no sistema de coordenadas original.

   R3.a) Com base nas análises feitas nas alíneas anteriores, escolha o melhor modelo e visualize os centróides. Guarde a figura e comente o que observa.



4. **Visualização em 2D:** Esta questão deve ser abordada após fazer as alíneas referentes ao PCA. Faça uma projeção 2D dos centróides do modelo escolhido na alínea anterior e dos dados.

   R4.a) Faça um gráfico da projeção obtida, distinguindo os centróides dos restantes pontos. Pinte os pontos de acordo com o centróide a que foram associados. O que pode dizer sobre os clusters? Como se relacionam com as visualizações feitas na questão anterior?

Parte II - PCA e Redução de Dimensionalidade

Esta secção tem como propósito a exploração do PCA como técnica de redução de dimensionalidade e exploração preliminar de um conjunto de dados. Pretende-se também demonstrar como o PCA pode ser utilizado para visualizações de dados de "elevada"dimensão em 2D ou 3D.

5. **Decomposição:** Usando a função `PCA` da biblioteca `scikit-learn`, faça a decomposição da matriz de poses X. Não defina o número de componentes nem o threshold de variância explicada.

   R5.a) Quantas componentes são precisas para obter 95% da variância explicada? Explique os passos que executou para determinar este valor. Se necessário, pode incluir figuras de suporte.

   Preparação dos dados: as poses foram concatenadas numa matriz X com 66 características por amostra.
   Aplicou-se StandardScaler antes do PCA, centrando e escalando as características.

   Aplicação do PCA: utilizou-se PCA() sem especificar previamente o número de componentes nem um limiar de variância explicada.

   R5.b) Escreva código que permita inspecionar a contribuição de cada característica no espaço original, *i.e.*, os marcadores e as suas componentes (u, v), relativamente a cada componente estimada pelo PCA. Depois, inspecione as componentes identificadas na alínea anterior, reporte o top-10 das *features*/características que mais contribuem e os seus pesos e comente aquilo que observa. Consegue observar alguma relação com os centróides obtidos pelo *k-means* na questão **R3.a)**?

Para analisar a contibuição das características originais de cada componente principal, foram inspecionados os loadings das oito componentes identificadas na R5.a. Para cada componente, selecionaram-se as 10 características com maior valor absoluto do peso, permitindo identificar os marcadores e as coordenadas "u" e "v" com maior influência na respetiva componente.

Os resultados mostram que a primeira componente principal (PC1), responsável por 39,03% da variância total, é dominada pelas coordenadas "v" de vários marcadores, cujos pesos são predominatemente positivos. A segunda componente (PC2), que explica 23,81% da variância, apresenta uma contribuição elevada sobretudo das coordenadas "u". Em conjunto, estas 2 componentes explicam aproximadamente 62,84% da variância dos dados.

Nas componentes seguintes, a contribuição das características torna-se mais localizada. A PC3 destaca principalmente as coordenadas "u" dos marcadores 28, 32, 30, 26, apresentando também pesos negativos associados a outros marcadores. A PC4 evidencia sobretudo as coordenadas "v" dos marcadores 31, 29 e 27. As componentes PC5 a PC8 apresentam padrões adicionais de variação.

A presença de pesos com sinais opostos indica que determinadas características variam em direções contrárias ao longo da respetiva PC. Contudo, a interpretação física destes padrões depende da localização e do significado dos marcadores do esqueleto.

A relação entre estas componentes e os centróides obtidos pelo k-means na R3.a

6. **Redução de dimensionalidade:** Escreva código para projetar a matriz de dados de fisioterapia numa dimensão mais baixa r, usando r componentes principais. Implemente também uma função que permita reconstruir a matriz de dados a partir da baixa dimensão. Escreva o seu próprio código usando a biblioteca `numpy` e não use funções já feitas do `scikit-learn`.

   R6.a) Faça uma redução de dimensionalidade para $r = 2$. Visualize os dados nesta nova dimensão e descreva o que observa.

Foi aplicado o PCA, implementado com NumPy, para reduzir as 66 características originais a duas componentes principais, obtendo uma projeção de 8374 poses que explica 62,84% da variância total, o que dá evidência que a implementação está correta. Observa-se uma região central densa, da qual partem vários braços em diferentes direções, alguns terminando em concentrações de pontos, nomeadamente à esquerda, à direita e na região inferior. Estas concentrações poderão corresponder a poses frequentes, enquanto as zonas mais dispersas poderão refletir transições entre poses consecutivas. De acordo com a R5.b, a PC1 é influenciada sobretudo pelas coordenadas "v", enquanto a PC2 apresenta contribuições importantes das coordenadas "u". Apesar das várias regiões de concentração, a sobreposição dos pontos não permite determinar claramente o número de clusters, sendo necessário complementar esta análise com outras visualizações.

   R6.b) Qual a variância explicada pelas duas componentes e o erro quadrático médio de reconstrução das poses em X? Em que marcadores/posições é que o erro é maior? Porquê?

As duas componentes principais explicam 62,84% da variância total. O erro quadrático médio de reconstrução (MSE), calculado sobre todas as poses e as 66 características, é 0,002589 no espaço original e 0,371601 no espaço normalizado. Os maiores erros absolutos ocorrem nos marcadores 15 – 22, correspondentes aos pulsos e às mãos, destacando-se o marcador 19 (MSE de 0,018261). Em contraste, os marcadores 23 e 24, correspondentes às ancas, apresentam erros reduzidos. Na R5.b, as coordenadas dos marcadores dos membros superiores também surgem entre as características relevantes das componentes PC3, PC5, PC6 e PC8, sugerindo que parte da sua variabilidade não é captada pelas duas primeiras componentes. A comparação do erro relativo e do desvio padrão permitirá determinar se os maiores erros absolutos resultam sobretudo da amplitude de movimento ou de uma representação menos eficaz pelo PCA.

R6.c) Repita a projeção e a alínea anterior, mas agora escolhendo r de modo a que o número de componentes explique 95% da variância. Compare os resultados com os obtidos para $r = 2$.

R6.c) Com oito componentes principais, a projeção passa a ter dimensão (8374 x 8) e explica 95,25% da variância, face aos 62,84% obtidos com duas componentes. Embora preserve mais informação, deixa de ser diretamente visualizável num gráfico 2D.
O MSE diminui de 0,002589 para 0,000287 no espaço original e de 0,371601 para 0,047536 no espaço normalizado, correspondendo a reduções de aproximadamente 9 e 7,8 vezes. Esta diferença deve-se às distintas escalas dos dados em cada espaço.
O erro do marcador 19 diminui de 0,018261 para 0,001493 (cerca de 91,8%), e o marcador 20 passa a apresentar o maior erro. Apesar da melhoria, permanecem 4,75% da variância por explicar, que pertence principalmente aos marcadores 15 a 22 devido a serem os de maior erro.
Para concluir se os pulsos e as mãos (marcadores 15 – 22) melhoram mais do que os pés, é necessário comparar a redução dos erros entre essas regiões.

Parte III - Misturas de Gaussianas (GMMs)

Esta secção tem como objectivo a exploração de um algoritmo de clustering com uma base probabilística, combinando o mesmo com técnicas de redução de dimensionalidade. Esta parte deve ser abordada após terem sido resolvidas as questões relativas ao PCA.

7. **Estimar um GMM em dimensão reduzida:** Utilizando a projeção obtida na alínea **R6.c)** estime um GMM usando a função `GaussianMixture` do `scikit-learn`. Permita que se estimem as matrizes de covariância completas e defina um número de componentes para a mistura que considere apropriado.

   R7.a) Reporte o número de componentes escolhido e justifique.

R7.b) Utilize o PCA obtido na Parte II para visualizar em 2D os clusters obtidos com o GMM, à semelhança do que fez para o *k-means* na questão **R4.a)**. Expanda o código de visualização para permitir inspecionar também as covariâncias estimadas. Comente o que observa.

R7.c) Escreva uma função que permita visualizar no espaço original dos esqueletos as poses e respetivas covariâncias (por marcador), tendo por base o GMM estimado na dimensão reduzida. Faça uma figura das poses obtidas e compare os rsultados com os obtidos pelo *k-means*. Comente também aquilo que observa em termos de variância por marcador e pose.

8. **Escolher o número de componentes da mistura:** Um dos desafios das GMMs é a escolha do número de componentes. Nesta questão iremos explorar um algoritmo proposto por Figueiredo e Jain[‡] para estimar automaticamente o número de componentes.

   R8.a) Familiarize-se com o artigo de Figueiredo e Jain e explique, sucintamente, como é que os autores propõe estimar automaticamente e de forma não supervisionada o número de componentes de uma mistura. Discuta tambem como é que o método proposto se diferencia do algoritmo EM estudado nas aulas para as misturas de Gaussianas.

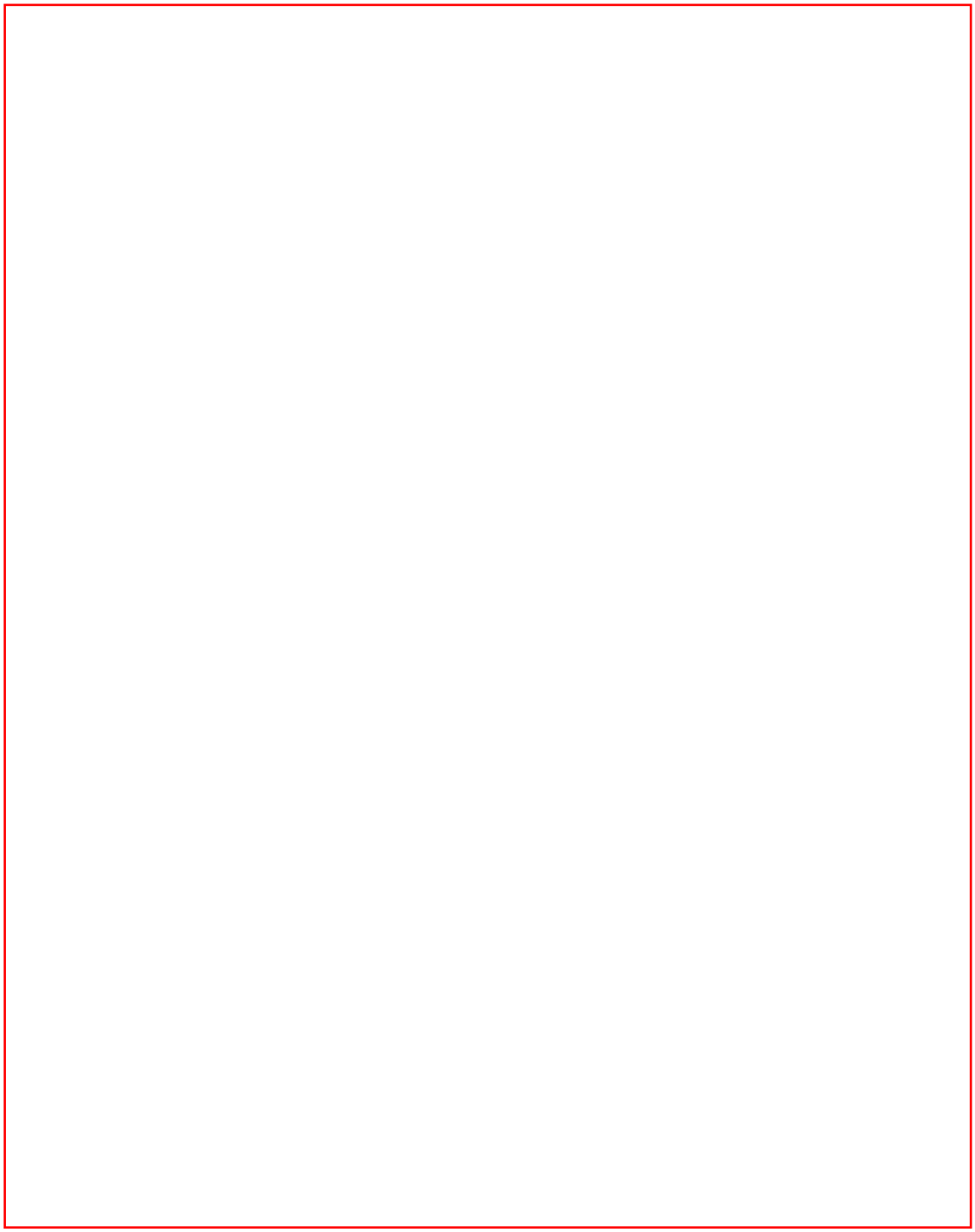
[‡]Figueiredo, Mario A. T., and Anil K. Jain. "Unsupervised learning of finite mixture models."IEEE Transactions on pattern analysis and machine intelligence 24, no. 3 (2002): 381-396.



Parte IV - Questões Abertas

Este bloco tem como objetivo promover a iniciativa e creatividade de cada grupo. O objetivo será tirar partido das várias ferramentas de aprendizagem não supervisionada estudadas nas aulas e abordadas nos laboratórios e tentar responder a duas questões sobre os dados recebidos.

9. **Número e tipo de exercícios:** Consegue estimar quantos exercícios diferentes estão representados nas sequências fornecidas sem visualizar as mesmas? E em que consistem os mesmos? Indique um número, tente descrever os exercícios, e explique claramente o procedimento adoptado. Todo o código e figuras utilizadas devem ser submetidos. **Use as ferramentas de aprendizagem não supervisionada - Não deve visualizar as sequências para responder!**



10. **Número de Pacientes:** Consegue estimar quantos pacientes fazem parte deste conjunto de dados? Pode assumir que todos os pacientes fazem todos os exercícios. Todavia, poderão apresentar diferentes níveis de constrangimento motor, o que afetará a forma como se movem. Adicionalmente, os pacientes podem ter corpos distintos. Indique um número e explique sucinta mas claramente o procedimento adoptado para chegar ao mesmo. Todo o código e figuras utilizadas devem ser submetidos.

A Declaração de Honra

Cada group deve preencher e assinar a seguinte declaração.

Declaração de Honra

Nós, membros do grupo [número do grupo], identificados abaixo

[Aluno 1], No. [xxxxxx]

[Aluno 2], No. [xxxxxx]

declaramos por nossa honra que este trabalho, realizado ao abrigo da cadeira de Introdução à Aprendizagem Automática é resultado do trabalho dos seguintes membros do grupop: members.

As contribuições individuais são:

[Aluno 1] – [breve descriçao da contribuição]

[Aluno 2] – [breve descriçao da contribuição]

Declaramos que este trabalho é original e que foi feito segundo os regulamentos académicos.

Nome, Número e	Assinatura
_____	_____
_____	_____
_____	_____

Data: _____

B Declaração do Uso de Inteligência Artifiial

Cada aluno deve preencher e assinar uma cópia individual desta declaração.

Declaração do Uso de Inteligência Artificial

Nome do Aluno: _____

Número do Aluno: _____

Declaro que todas as decisões e conteúdo técnico deste laboratório são da minha autoria.

- [listar ferramenta] para [finalidade]

Declaro que compreendo completamente o funcionamento de todas as componentes do projeto, mesmo as que foram desenvolvidas com o apoio das ferramentas de IA listadas acima."

Data: _____ Assinatura: _____